

Machine learning and deep learning

Msc Renzo Claure

1



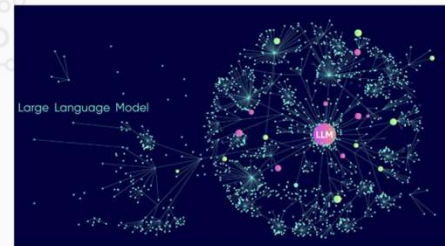
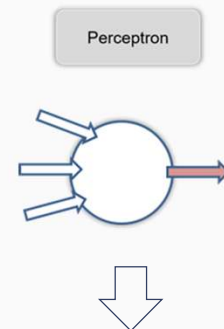
Redes Neuronales

Msc Renzo Claure

2

Breve Historia

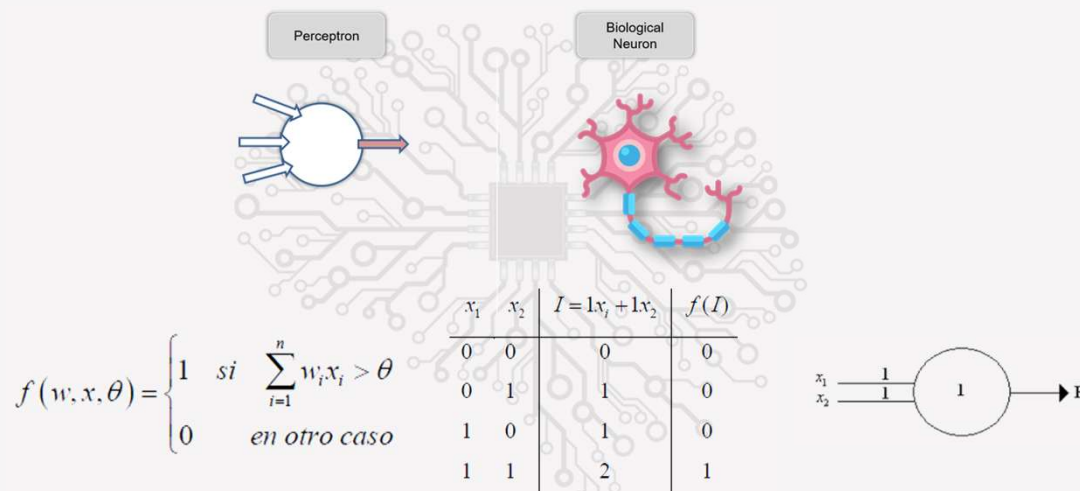
- 1943 McCulloch y Pitts: Modelo de neurona artificial
- 1949 Donald Hebb: Regla de Hebb
- 1958 Frank Rosenblatt: Perceptrón
- 1969 Minsky y Papert: Crítica al perceptrón ("invierno de la IA")
- 1986 Rumelhart, Hinton y Williams: Retropropagación
- 1989 Yann LeCun: Redes convolucionales (CNN)
- 1997 Hochreiter y Schmidhuber: LSTM (Redes recurrentes)
- 2006 Geoffrey Hinton: Aprendizaje profundo (Deep Learning)
- 2012 AlexNet: Victoria en ImageNet (Deep Learning moderno)
- 2014 Ian Goodfellow: GANs (Redes generativas adversarias)
- 2017 Transformers (Revolución en NLP)
- 2020s GPT-3, GPT-4: Modelos de lenguaje a gran escala



Msc Renzo Claire

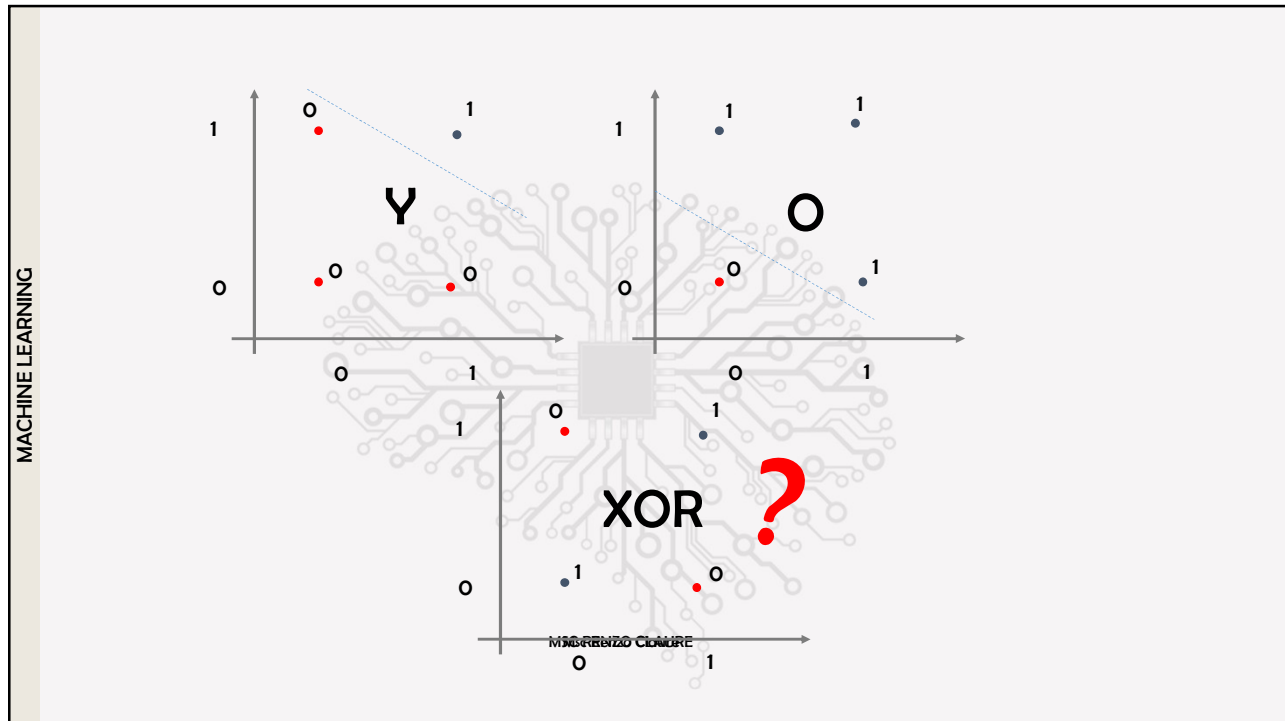
3

El perceptron y la neurona



Msc Renzo Claire

4



5

MACHINE LEARNING

Tipos de redes neuronales	
Redes Neuronales Feedforward: Son las redes neuronales más básicas, donde la información fluye en una sola dirección, desde la entrada hasta la salida, sin ciclos o bucles.	
Redes Neuronales Convolucionales CNN: Especializadas en el procesamiento de datos con una estructura de cuadrícula, como imágenes. Utilizan capas convolucionales para extraer características espaciales.	
Redes Neuronales Recurrentes Memoria a Corto-Largo Plazo: Un tipo especial de RNN que puede aprender dependencias a largo plazo. Utilizan células de memoria para mantener información durante largos períodos.	
Transformers: Arquitectura basada en mecanismos de atención que ha revolucionado el procesamiento de lenguaje natural. No utiliza recurrencia, lo que permite un procesamiento más paralelizable.	
Redes Neuronales Generativas Adversariales GAN: Consisten en dos redes (un generador y un discriminador) que compiten entre sí para generar datos realistas.	
Redes Neuronales de Autoencoders: Redes que aprenden una representación comprimida de los datos de entrada. Consisten en un codificador y un decodificador.	
Redes Neuronales de Grafos Diseñadas para trabajar con datos estructurados en forma de grafos, donde los nodos y las aristas representan entidades y relaciones.	
Redes Neuronales de Aprendizaje por Refuerzo, Redes Neuronales de Capsulas, Redes Neuronales de Memoria, Redes Neuronales de Aprendizaje Profundo Residual, Redes Neuronales de Aprendizaje Profundo Densamente Conectadas, Redes Neuronales de Aprendizaje Profundo Espacial-Temporal,	
Msc Renzo Claire	

6



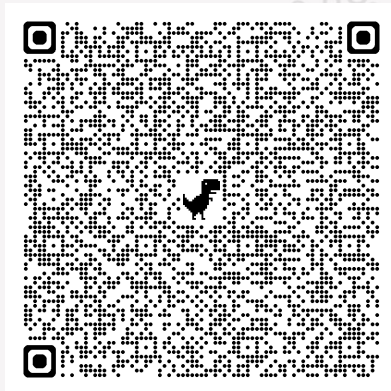
Redes Neuronales MLP

fundamentos

Msc Renzo Claure

7

Playground MLP

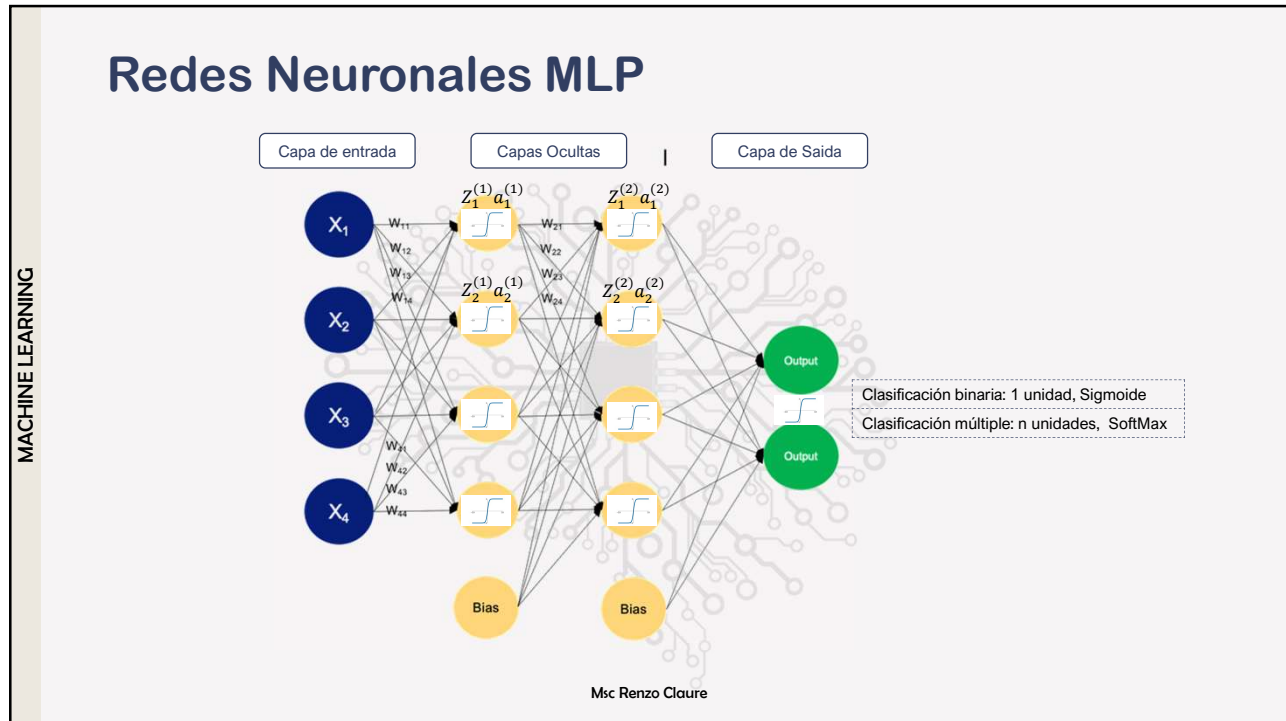


MACHINE LEARNING

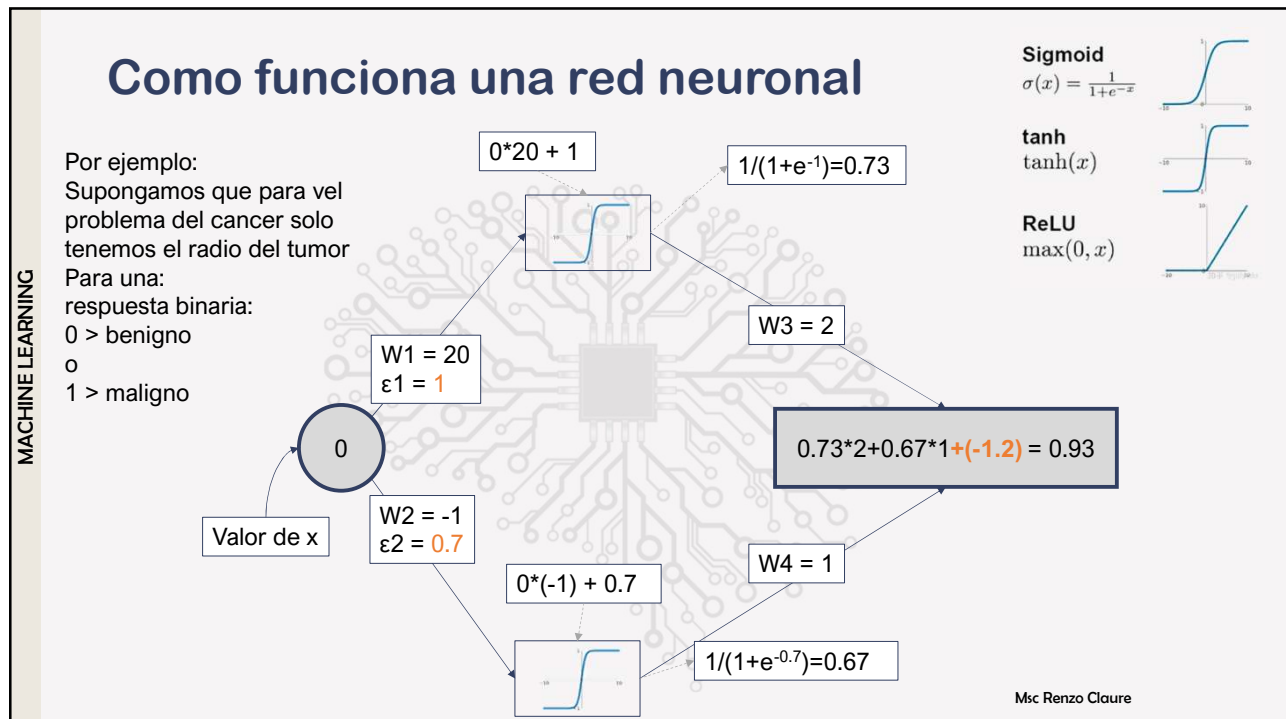
[TensorFlow PLayerGround](#)

Msc Renzo Claure

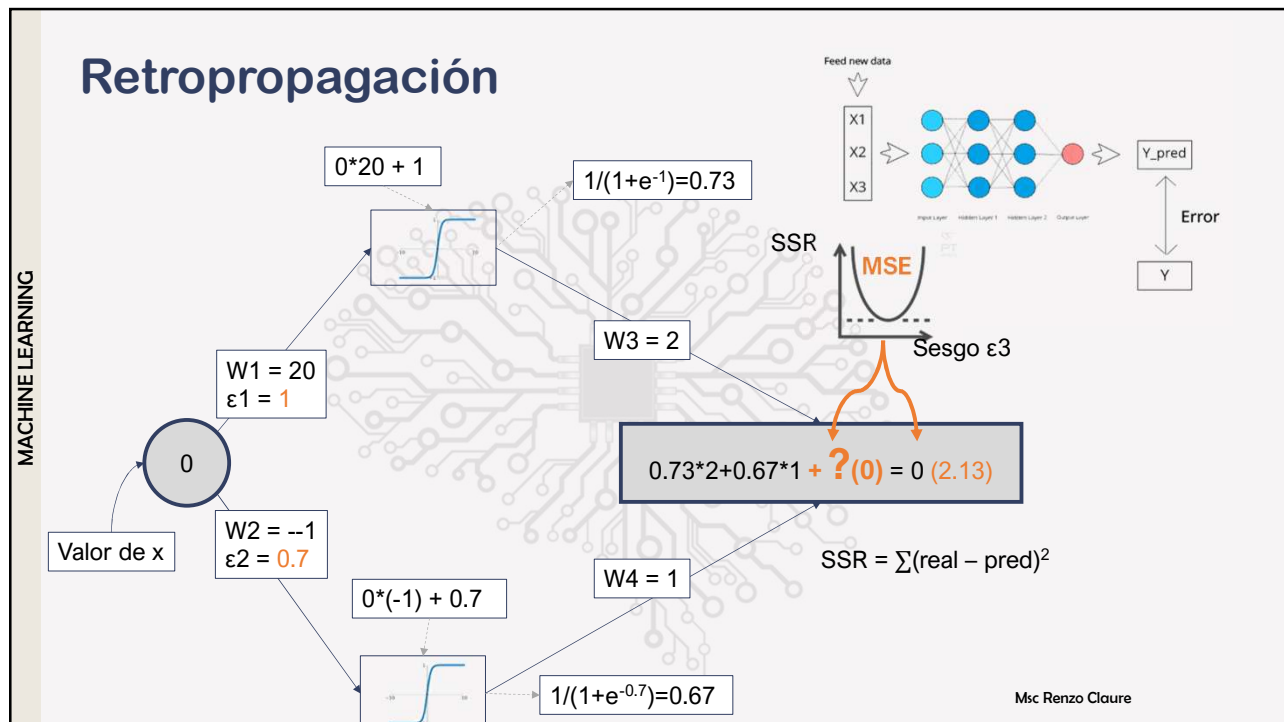
8



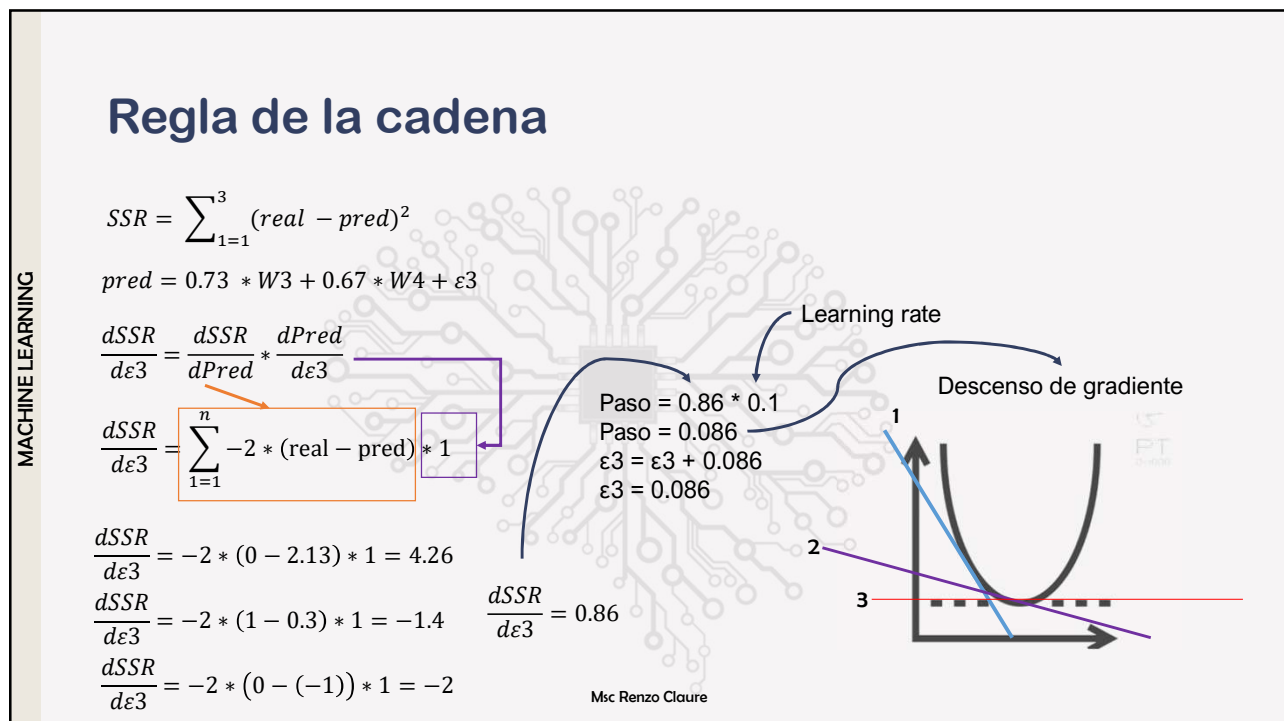
9



10



11

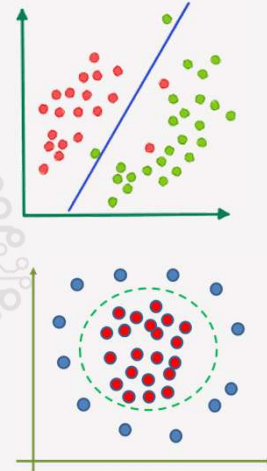


12

Funciones de activación

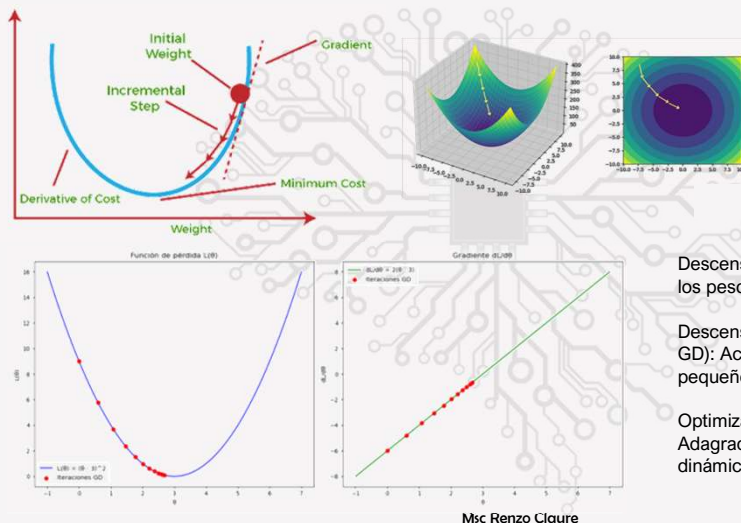
ReLU $\max(0, x)$	GELU $\alpha \left(\frac{1}{\sqrt{2}} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + \alpha x^3) \right) \right) \right)$	PReLU $\max(0, x)$
ELU $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish $x \cdot \frac{1}{1 + \exp(-x)}$	SELU $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish $x \tanh \left(\frac{1}{3} \log(1 + \exp(\beta x)) \right)$	RReLU $\begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \text{ with } \alpha \sim \mathcal{U}(l, u) \end{cases}$
HardSwish $\begin{cases} 0 & \text{if } x < -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid $\frac{1}{1 + \exp(-x)}$	SoftSign $\frac{x}{1 + x }$
Tanh $\tanh(x)$	Hard tanh $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink $x - \tanh(x)$	Soft Shrink $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

Msc Renzo Claire



13

Descenso de gradiente



Msc Renzo Claire

Descenso de gradiente estocástico (SGD): Actualiza los pesos después de cada muestra.

Descenso de gradiente por mini-lotes (Mini-batch GD): Actualiza los pesos después de procesar un pequeño grupo de muestras.

Optimizadores avanzados: Adam, RMSprop, Adagrad, etc., que adaptan la tasa de aprendizaje dinámicamente para mejorar la convergencia.

14

Redes neuronales clásicas (Vanila)

Principales parámetros en ScikitLearn

- Hidden layer sizes: configura el número de capas ocultas, y el número de nodos o elemento en cada capa, (por defecto 100)
- Activacion: Configura la función de activación no lineal (rectificada lineal uniforme RELU, tanh Hiperbólica tangencial, logística)
- Alpha: controla el factor de penalización para la regularización (elimina la variable), por defecto es: 0,0001 (L2)
- Solver: Configura el algoritmo de aprendizaje de los pesos, busca los pesos óptimos. Adam es el usado por defecto y es bueno para grandes datasets, para sets pequeños como en los ejemplos "lbfgs" es más efectivo

Msc Renzo Claire

15

Redes neuronales clásicas (Vanila)

ventajas y desventajas

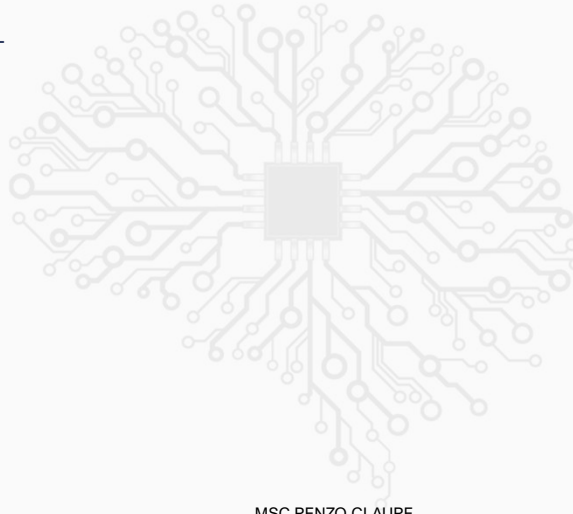
- Puede descubrir interacciones complejas entre las variables
- Muy utilizado en aplicaciones de tiempo real
- Son las que más se están desarrollando, el término Deep-learning hace referencia a RN más complejas
- Requiere bastantes datos, más tiempo de preparación y entrenamiento
- Es una buena elección cuando las variables son similares, por ejemplo en reconocimiento facial. No son muy efectivas cuando las variables son muy distintas

MSC RENZO CLAIRE

16

Redes neuronales MLP (Vanila)

- NB_19_MLP_SKL



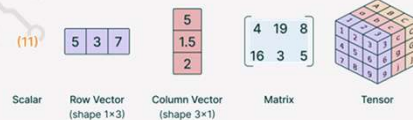
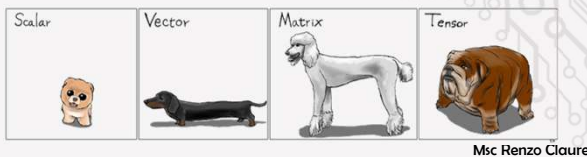
MSC RENZO CLAURE

17

Tensorflow



- TensorFlow es una biblioteca de software de código abierto desarrollada por Google para crear y entrenar modelos de aprendizaje automático y aprendizaje profundo.
- TensorFlow utiliza tensores porque son estructuras de datos fundamentales que permiten representar y manipular información de forma eficiente:
 - Representación Multidimensional
 - Optimización en Hardware Especializado
 - Construcción de Grafos Computacionales
 - Flexibilidad y Abstracción



18



- Keras es una API de redes neuronales de alto nivel, escrita en Python. Se centra en la facilidad de uso y la rapidez en el desarrollo de modelos de aprendizaje profundo. Originalmente, Keras podía ejecutarse sobre diferentes backends, como TensorFlow, Theano y CNTK. Sin embargo, ahora está estrechamente integrado con TensorFlow (tf.keras), convirtiéndose en la API de alto nivel recomendada para TensorFlow.
- Facilidad de uso.
- Modularidad.
- Extensibilidad.
- Integración con TensorFlow

```
# Definición del modelo a bajo nivel
class LowLevelModel(tf.Module):
    def __init__(self):
        # Inicialización de pesos y sesgos para la capa oculta
        self.W1 = tf.Variable(tf.random.normal([input_dim, hidden_units]), name="W1")
        self.b1 = tf.Variable(tf.zeros([hidden_units]), name="b1")
        # Inicialización de pesos y sesgos para la capa de salida
        self.W2 = tf.Variable(tf.random.normal([hidden_units, 1]), name="W2")
        self.b2 = tf.Variable(tf.zeros([1]), name="b2")

    def __call__(self, x):
        # Capa oculta: cálculo lineal + activación ReLU
        z1 = tf.matmul(x, self.W1) + self.b1
        a1 = tf.nn.relu(z1)
        # Capa de salida: cálculo lineal + activación Sigmoid
        z2 = tf.matmul(a1, self.W2) + self.b2
        output = tf.nn.sigmoid(z2)
        return output

model_low = LowLevelModel()
x_sample = tf.random.normal([1, input_dim])
y_pred_low = model_low(x_sample)
print("Predicción con TensorFlow low-level:", y_pred_low)
```

```
# Parámetros del modelo
input_dim = 10 # Dimensión de la entrada
hidden_units = 16 # Número de neuronas en la capa oculta

# Definición del modelo usando Keras (API de alto nivel)
model_keras = Sequential([
    Dense(hidden_units, input_dim=input_dim, activation='relu'),
    Dense(1, activation='sigmoid')
])

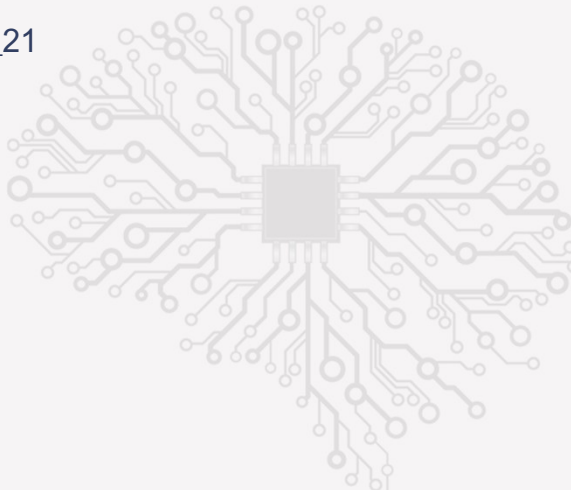
y_pred_keras = model_keras(x_sample)
print("Predicción con Keras:", y_pred_keras)
```

Msc Renzo Claire

19

Tensorflow

- NB_20 y NB_21



Msc Renzo Claire

20

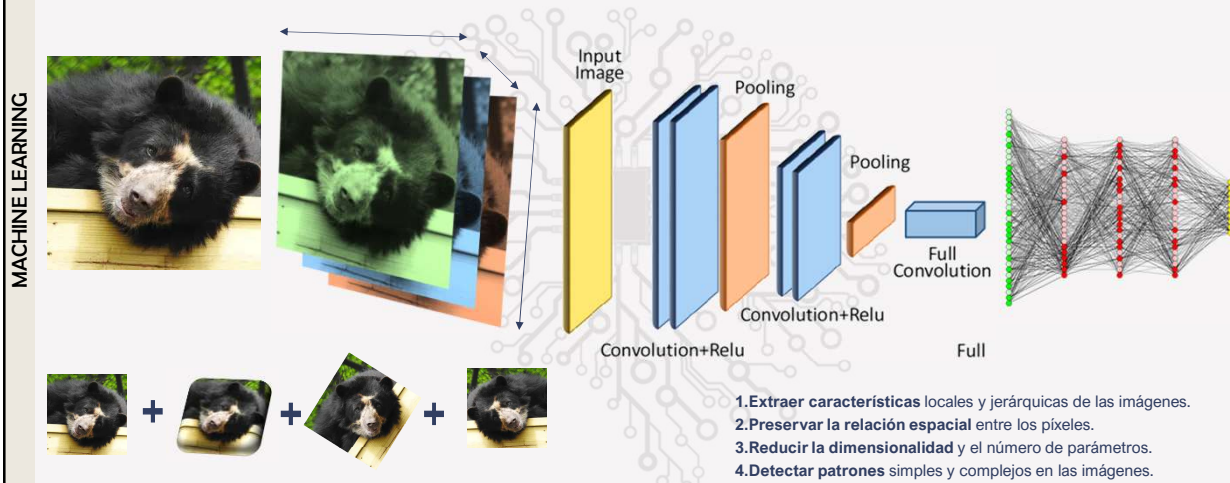


Redes convolucionales

Msc Renzo Claire

21

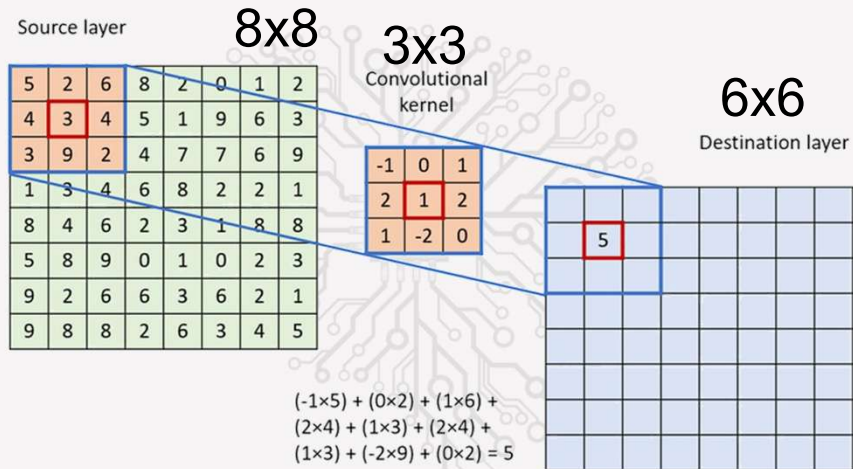
Redes neuronales convolucionales CNN



Msc Renzo Claire

22

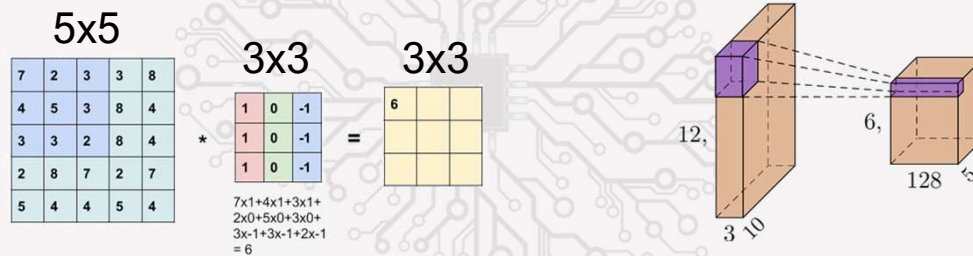
Convoluciones



Msc Renzo Claire

23

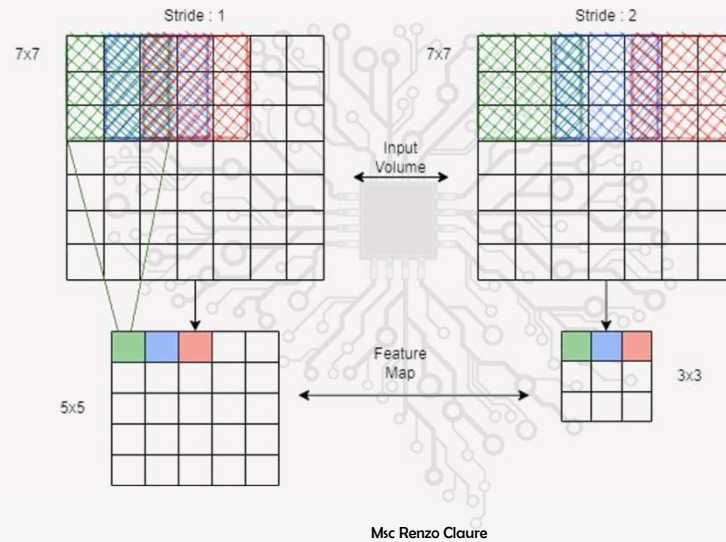
Convoluciones



Msc Renzo Claire

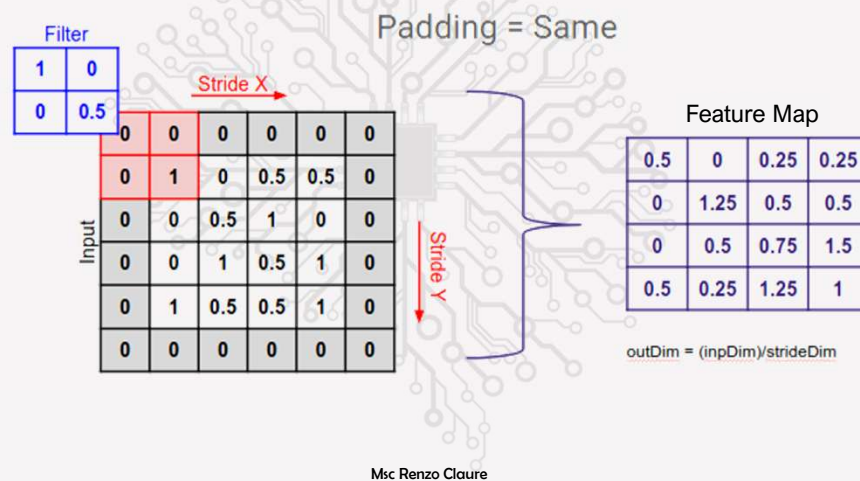
24

Operaciones Stride



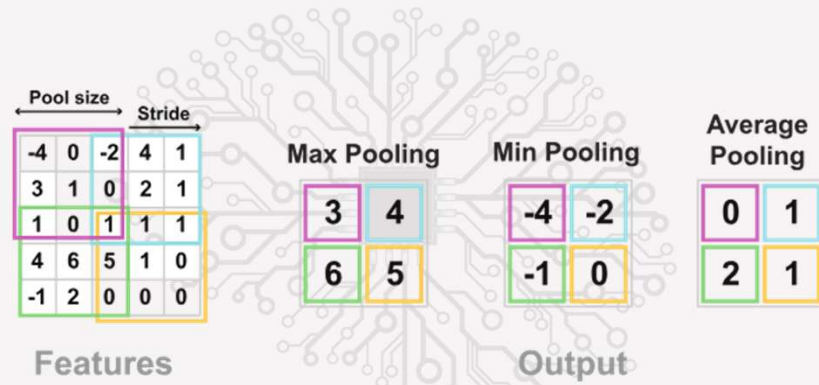
25

Operaciones Padding



26

Operaciones Pooling



Msc Renzo Claire

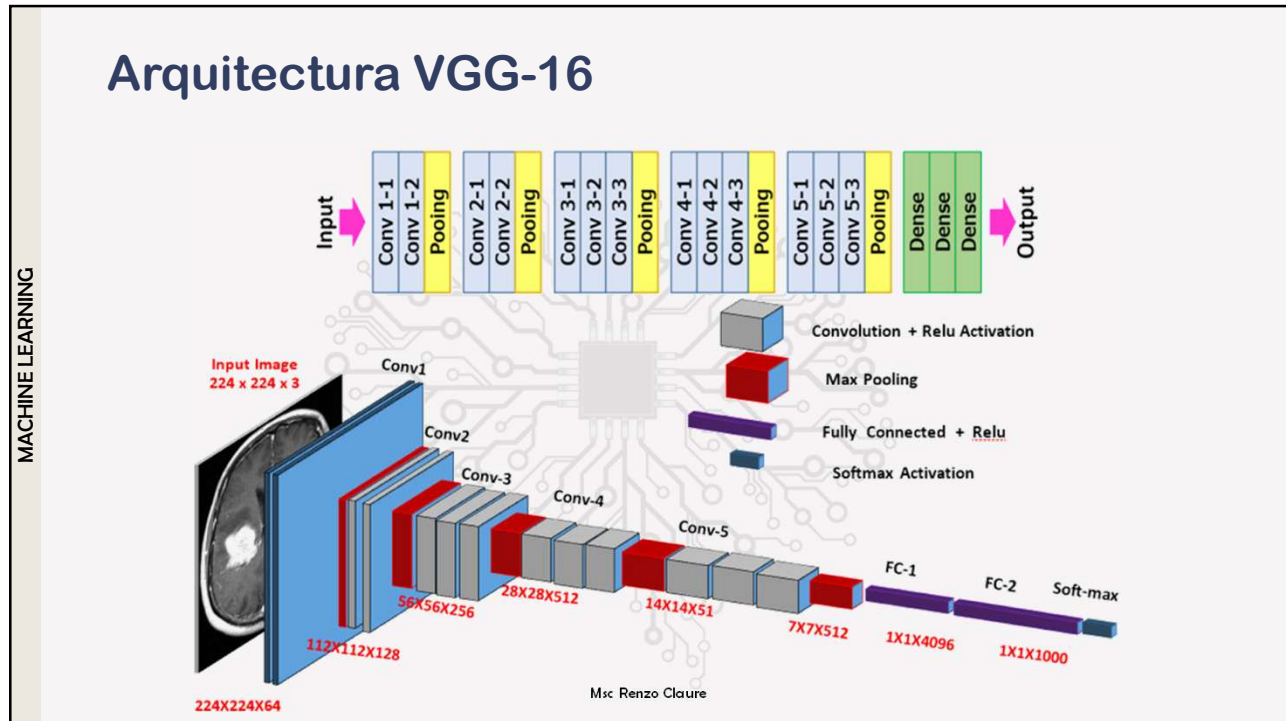
27

Arquitecturas de CNN

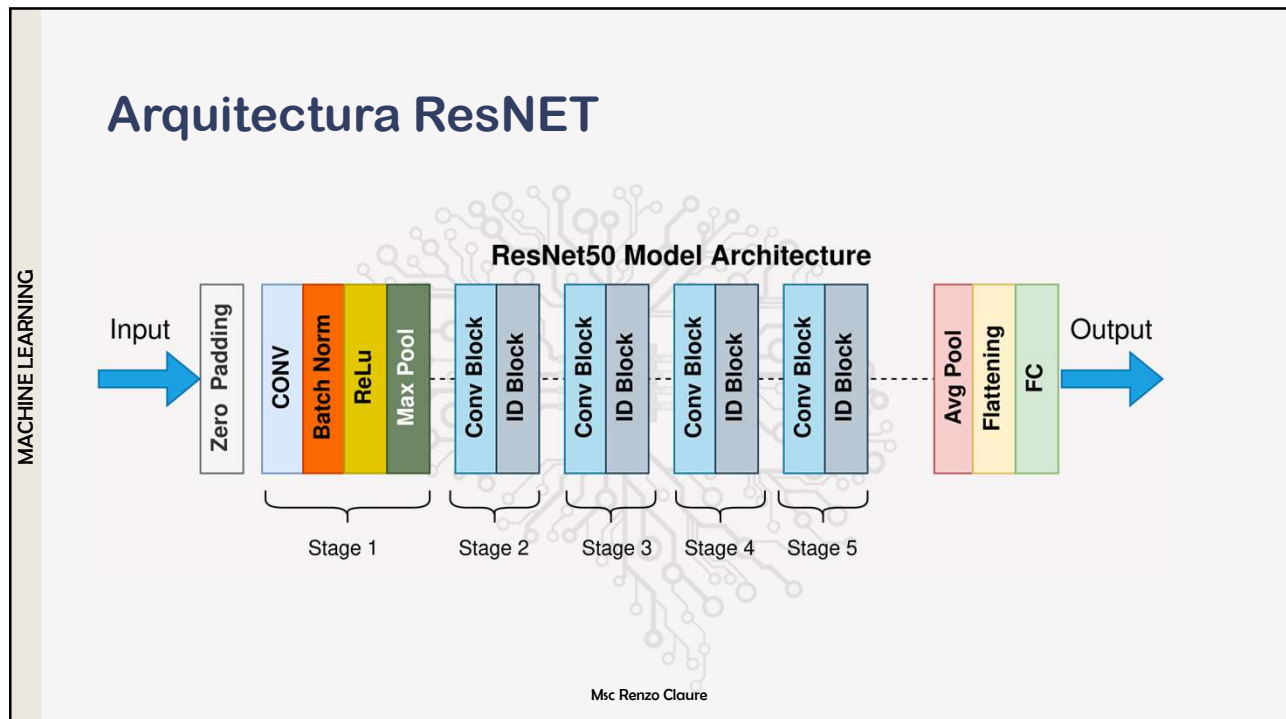
Arquitectura	Año	Profundidad	Contribución clave	Ventajas	Aplicaciones principales
LeNet-5	1998	5 capas	Primera CNN exitosa para reconocimiento de dígitos.	Simple y eficiente para tareas pequeñas.	Reconocimiento de caracteres.
AlexNet	2012	8 capas	Popularizó las CNN al ganar ImageNet 2012.	Uso de ReLU y dropout para mejorar rendimiento.	Clasificación de imágenes.
VGGNet	2014	16-19 capas	Demostó la importancia de la profundidad.	Bloques convolucionales simples (3x3).	Extracción de características, transfer learning.
GoogLeNet	2014	22 capas	Introdujo el módulo Inception.	Efficiente en parámetros y computación.	Clasificación y detección de objetos.
ResNet	2015	50-152 capas	Introdujo conexiones residuales.	Permite entrenar redes muy profundas.	Clasificación, detección y segmentación.
DenseNet	2017	Variable	Conexiones densas entre capas.	Mejor flujo de gradientes, menos parámetros.	Clasificación y detección de objetos.
MobileNet	2017	Variable	Convoluciones separables en profundidad.	Ligero y eficiente para dispositivos móviles.	Aplicaciones en tiempo real en móviles.
EfficientNet	2019	Variable	Escalado compuesto (profundidad, ancho, resolución).	Alto rendimiento con menos parámetros.	Clasificación, detección y segmentación.
SqueezeNet	2016	Variable	Fire modules para reducir parámetros.	Extremadamente ligero (<1M parámetros).	Aplicaciones en dispositivos limitados.
Xception	2017	Variable	Convoluciones separables extremas.	Mejora la eficiencia de las convoluciones.	Clasificación y transfer learning.
NASNet	2018	Variable	Búsqueda automática de arquitecturas (NAS).	Optimizada automáticamente para un dataset.	Clasificación y detección de objetos.

Msc Renzo Claire

28



29

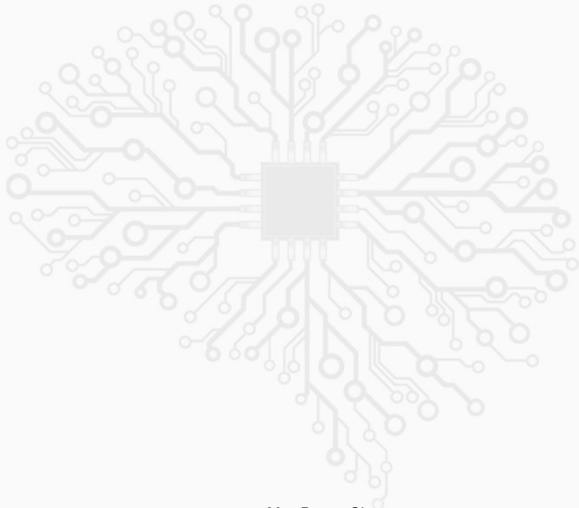


30

MACHINE LEARNING

CNN

- NB_CNN



Msc Renzo Claure

31

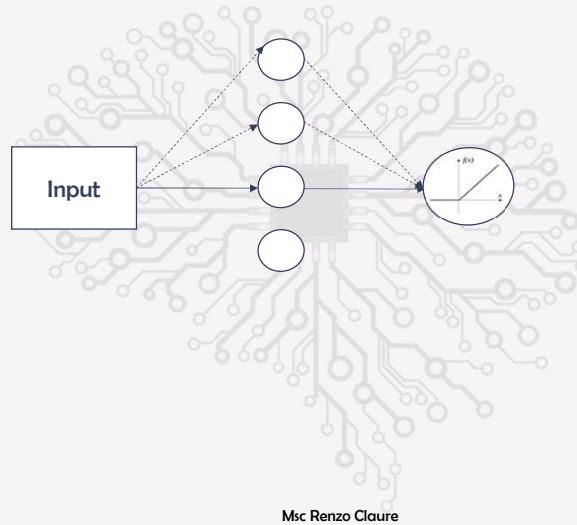


RNN LSTM

Msc Renzo Claure

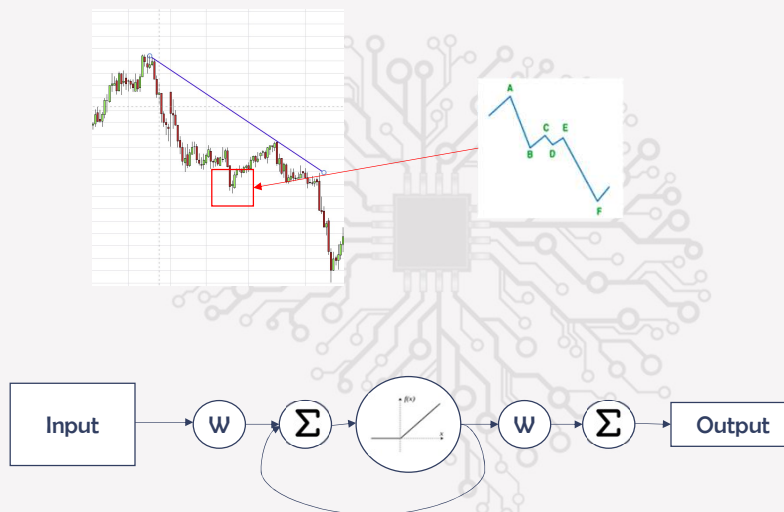
32

Red Neuronal Multiapa (MLP)



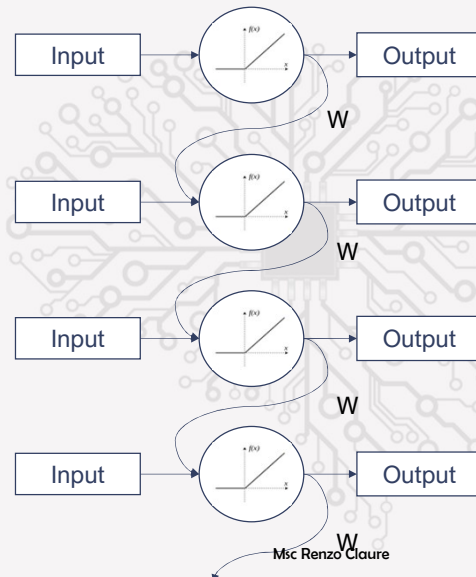
33

Red Neuronal Recurrente (RNN)



34

Red Neuronal Recurrente (RNN)



$$W^n$$

Si tenemos datos históricos de tres meses, digamos noventa días.

Entonces si $W = 2$:

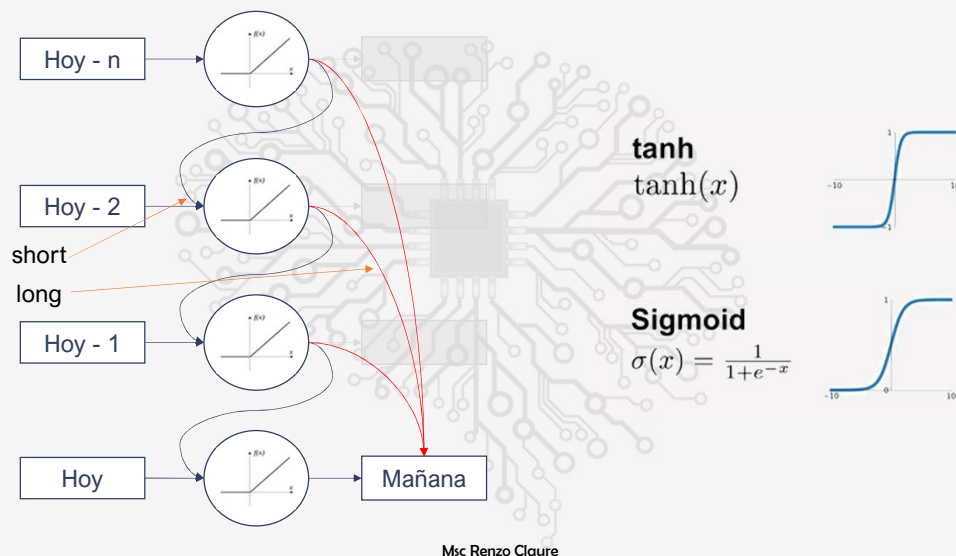
$W^{90} = 2^{90} \rightarrow \text{inf.}$

Entonces si $W = 0.5$:

$W^{90} = 0.5^{90} \rightarrow 0$

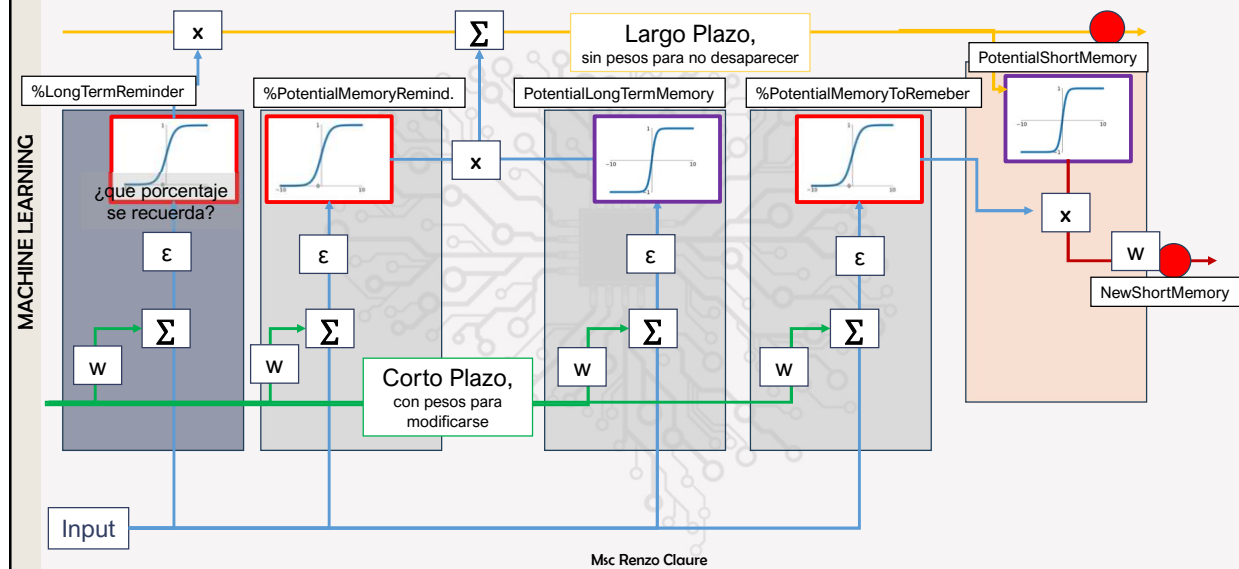
35

Red Neuronal Recurrente (RNN) con Long Short Term Memory (LSTM)



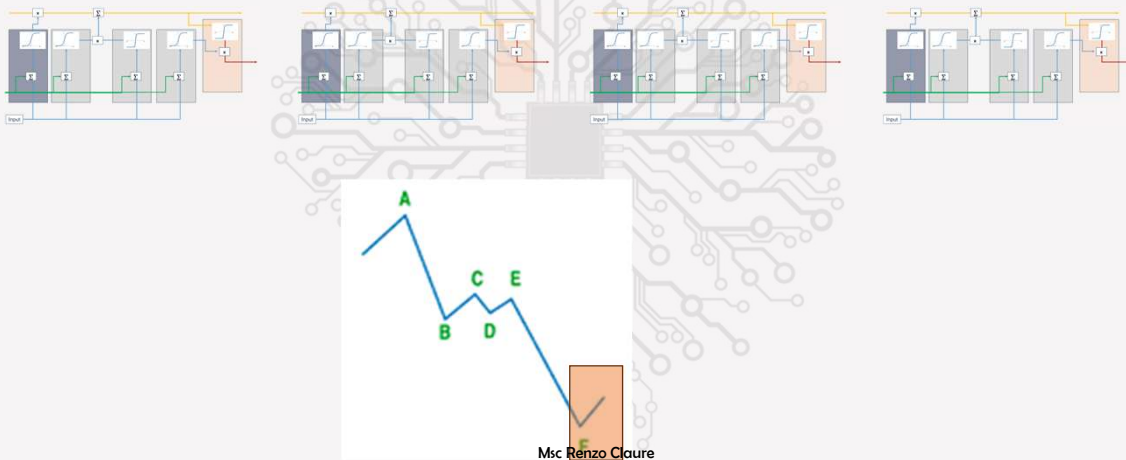
36

MACHINE LEARNING



Msc Renzo Claure

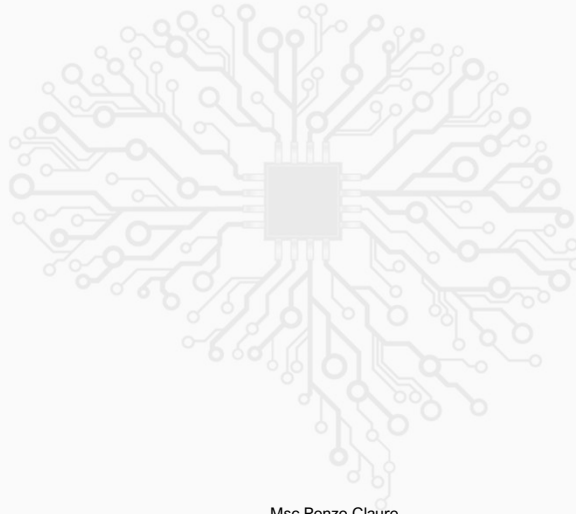
MACHINE LEARNING



Msc Renzo Claure

Red Neuronal Recurrente (RNN) con Long Short Term Memory (LSTM)

- NB_



Msc Renzo Claure